Module – 4

8051 Interrupts

SYLLABUS:

Interrupt Programming: Basics of Interrupts, 8051 Interrupts, Programming Timer Interrupts, Programming Serial Communication Interrupts, Interrupt Priority in 8051(Assembly Language only) **(Text book 2- 3.6, Text book 1- 11.1,11.2,11.4, 11.5)**

Table of Contents

TEXT, REFERENCE & ADDITIONAL REFERENCE BOOKS

	BOOK TITLE/AUTHORS/PUBLICATION/LINK
T-1	"The 8051 Microcontroller and Embedded Systems - Using Assembly and C", Muhammad Ali Mazidi and Janice Gillespie Mazidi and Rollind. Mckinlay; Phi, 2006 / Pearson, 2006.
T-1	"The 8051 Microcontroller", Kenneth j. Ayala, 3rd edition, Thomson/Cengage Learning.
T-1	"Programming and Customizing The 8051 Microcontroller", Myke Predko, Tata Mc Graw-Hill Edition 1999 (reprint 2003).
R-1	"The 8051 Microcontroller Based Embedded System", Manish K Patel, McGraw Hill, 2014, ISBN: 978-93-329-0125-4.
R-2	Microcontrollers: Architecture, Programming, Interfacing and System Design", Raj Kamal, Pearson Education, 2005

4.1 Interrupts

Interrupt is one of the most important and powerful concepts and features in microcontroller/processor applications. Almost all the real world and real time systems built around microcontrollers and microprocessors make use of interrupts.

4.1.1 What is an Interrupt?

An interrupt refers to a notification, communicated to the controller, by a hardware device or software, on receipt of which controller momentarily stops and responds to the interrupt. Whenever an interrupt occurs the controller completes the execution of the current instruction and starts the execution of an **Interrupt Service Routine** (ISR) or Interrupt Handler.

ISR is a piece of code that tells the processor or controller what to do when the interrupt occurs. After the execution of ISR, controller returns back to the instruction it has jumped from (before the interrupt was received). The interrupts can be either **hardware interrupts** or **software interrupts**.

4.1.2 Need of interrupts

An application built around microcontrollers generally has the following structure. It takes input from devices like keypad, ADC etc.; processes the input using certain algorithm; and generates an output which is either displayed using devices like seven segment, LCD or used further to operate other devices like motors etc. In such designs, controllers interact with the inbuilt devices like timers and other interfaced peripherals like sensors, serial port etc. The programmer needs to monitor their status regularly like whether the sensor is giving output, whether a signal has been received or transmitted,whether timer has finished counting, or if an interfaced device needs service from the controller, and so on. This state of continuous monitoring is known as polling.

In polling, the microcontroller keeps checking the status of other devices; and while doing so it does no other operation and consumes all its processing time for monitoring. This problem can be addressed by using interrupts. In interrupt method, the controller responds to only when an interruption occurs. Thus, in interrupt method, controller is not required to regularly monitor the status (flags, signals etc.)of interfaced and inbuilt devices.

To understand the difference better, consider the following. The polling method is very much similar to a salesperson. The salesman goes door-to-door requesting to buy its product or service. Like controller keeps monitoring the flags or signals one by one for all devices and caters to whichever needs its service. Interrupt, on the other hand, is very similar to a shopkeeper. Whosoever needs a service or product goes to him and apprises him of his/her needs. In our case, when the flags or signals are received, they notify the controller that they need its service.

4.1.3 Hardware & Software Interrupt

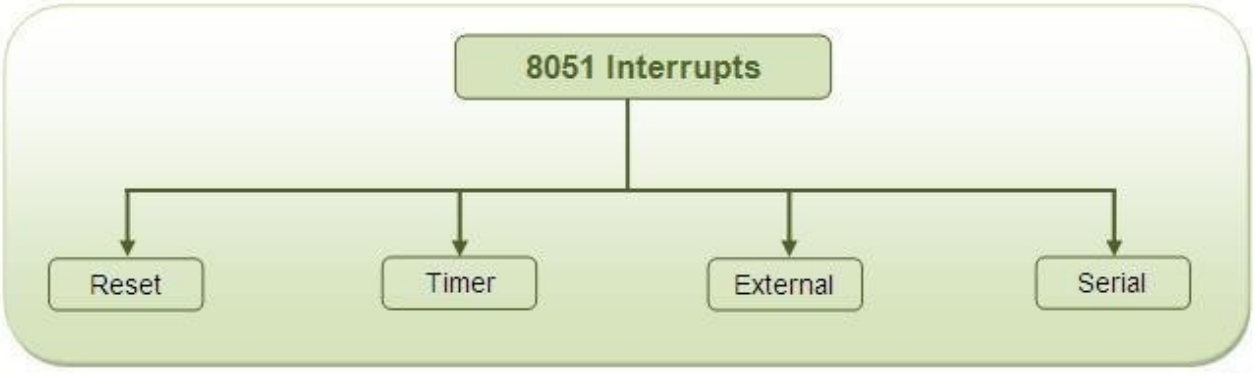
The interrupts in a controller can be either hardware or software. If the interrupts are generated by the controller's inbuilt devices, like timer interrupts; or by the interfaced devices, they are called the hardware interrupts. If the interrupts are generated by a piece of code, they are termed as software interrupts.

Multiple interrupts

What would happen if multiple interrupts are received by a microcontroller at the same instant? In sucha case, the controller assigns priorities to the interrupts. Thus, the interrupt with the highest priority is served first. However, the priority of interrupts can be changed configuring the appropriate registers in the code.

4.2 8051 Interrupts

The 8051 controller has six hardware interrupts of which five are available to the programmer. These are as follows:



1. **RESET interrupt** – This is also known as Power on Reset (POR). When the RESET interrupt isreceived, the controller restarts executing code from 0000H location. This is an interrupt which is not available to or, better to say, need not be available to the programmer.

2. **Timer interrupts** – Each Timer is associated with a Timer interrupt. A timer interrupt notifies themicrocontroller that the corresponding Timer has finished counting.

3. **External interrupts** – There are two external interrupts EX0 and EX1 to serve external devices. Both these interrupts are active low. In AT89C51, P3.2 (INT0) and P3.3 (INT1) pins are available for external interrupts 0 and 1 respectively. An external interrupt notifies the microcontroller that an external device needs its service.

4. **Serial interrupt** – This interrupt is used for serial communication. When enabled, it notifies the controller whether a byte has been received or transmitted.

4.2.1 How is an interrupt serviced?

Every interrupt is assigned a fixed memory area inside the processor/controller. The Interrupt Vector Table (IVT) holds the starting address of the memory area assigned to it (corresponding to every interrupt).

The interrupt vector table (IVT) for AT89C51 interrupts is as follows :

Interrupt	ROM Location (Hex)	Pin	Flag clearing
Reset	0000	9	Auto

External interrupt 0	0003	12	Auto
Timer interrupt 0	000B	–	Auto
External interrupt 1	0013	13	Auto
Timer interrupt 1	001B	–	Auto
Serial COM interrupt	0023	–	Programmer clears it

When an interrupt is received, the controller stops after executing the current instruction. It transfers the content of program counter into stack. It also stores the current status of the interrupts internally but not on stack. After this, it jumps to the memory location specified by **Interrupt Vector Table** (IVT). After that the code written on that memory area gets executed. This code is known as the Interrupt Service Routine (ISR) or interrupt handler. ISR is a code written by the programmer to handle or service the interrupt.

4.3 Programming Interrupts

While programming interrupts, first thing to do is to specify the microcontroller which interrupts must be served. This is done by configuring the Interrupt Enable (IE) register which enables or disables the various available interrupts. The Interrupt Enable register has following bits to enable/disable the hardware interrupts of the 8051 controller.

EA	-	ET2	ES	ET1	EX1	ET0	EX0

Bit Values of IE Register of 8051 Microcontroller

To enable any of the interrupts, first the EA bit must be set to 1. After that the bits corresponding to the desired interrupts are enabled. ET0, ET1 and ET2 bits are used to enable the Timer Interrupts 0,1 and 2, respectively. In AT89C51, there are only two timers, so ET2 is not used. EX0 and EX1 are used to enable the external interrupts 0 and 1. ES is used for serial interrupt.

EA bit acts as a lock bit. If any of the interrupt bits are enabled but EA bit is not set, the interrupt will not function. By default all the interrupts are in disabled mode.

Note that the IE register is bit addressable and individual interrupt bits can also be accessed.
For example – IE = 0x81; enables External Interrupt0 (EX0)IE = 0x88; enables Serial Interrupt

Example 1

Show the instructions to (a) enable the serial interrupt, timer 0 interrupt, and external hardware interrupt 1 (EX1),and (b) disable (mask) the timer 0 interrupt, then (c) show how to disable all the interrupts with a single instruction.

Solution:

(a) MOV IE,#10010110B ;enable serial, timer 0, EX1

Another way to perform the same manipulation is

```
        SETB IE.7   ;EA=1, global enable
        SETB IE.4   ;enable serial interrupt
        SETB IE.1   ;enable Timer 0 interrupt
        SETB IE.2   ;enable EX1
(b)     CLR  IE.1    ;mask (disable) timer 0 interrupt only
(c)     CLR  IE.7    ;disable all interrupts
```

4.4 Timer Interrupt:

Programming Timer Interrupts

The timer interrupts T0 and T1 are related to Timers 0 and 1, respectively. The interrupt programming for timers involves following steps :

1. Configure TMOD register to select timer(s) and its/their mode.
2. Load initial values in THx and TLx for mode 0 and 1; or in THx only for mode 2.
3. Enable Timer Interrupt by configuring bits of IE register.
4. Start timer by setting timer run bit TRx.
5. Write subroutine for Timer Interrupt.
6. Note that it is not required to clear timer flag TFx.
7. To stop the timer, clear TRx in the end of subroutine.
8. If the Timer has to run again and again, it is required to reload initial values within the routine itself

Example 2

Write a program that continuously get 8-bit data from P0 and sends it to P1 while simultaneously creating a square wave of 200 us period on pin P2.1. Use timer 0 to create the square wave. Assume that XTAL = 11.0592 MHz.

Solution:

We will use timer 0 in mode 2 (auto reload). TH0 = 100/1.085 us = 92

```
ORG  0000H
LJMP MAIN      ;by-pass interrupt vector table

;ISR for timer 0 to generate square wave
ORG  000BH      ;Timer 0 interrupt vector table
CPL  P2.1        ;toggle P2.1 pin
RETI             ;return from ISR

;The main program for initialization
```

```
        ORG  0030H              ;after vector table space
MAIN: MOV   TMOD,#02H     ;Timer 0, mode 2
MOV   P0,#0FFH            ;make P0 an input port
MOV   TH0,#-92            ;TH0=A4H for -92
MOV  IE,#82H              ;IE=10000010 (bin) enable Timer 0
SETB TR0                  ;Start Timer 0
BACK: MOV   A,P0          ;get data from P0
MOV  P1,A                 ;issue it to P1
SJMP BACK                 ;keep doing it loop unless interrupted by TF0
END
```

Example 3

Write a program to generate a square wave if 50kHz frequency on pin P1.2. Assume that XTAL=11.0592 MHz

Solution:

```
ORG 0
LJMP MAIN

ORG 000BH     ;ISR for Timer 0
CPL P1.2
RETI

ORG 30H
;--------main program for initialization
MAIN:MOV TM0D,#00000001B       ;Timer 0, Mode 1
MOV TL0,#00
MOV TH0,#0DCH
MOV IE,#82H                    ;enable Timer 0 interrupt
SETB TR0
HERE:SJMP HERE
END
```

4.5 Serial Communication Interrupt

- TI (transfer interrupt) is raised when the last bit of the framed data, the stop bit, is transferred, indicating that the SBUF register is ready to transfer the next byte
- RI (received interrupt) is raised when the entire frame of data, including the stop bit, is received
- In other words, when the SBUF register has a byte, RI is raised to indicate that the received byte needs to be picked up before it is lost (overrun) by new incoming serial data

- In the 8051 there is only one interrupt set aside for serial communication. This interrupt is used to both send and receive data
- If the interrupt bit in the IE register (IE.4) is enabled, when RI or TI is raised the 8051 gets interrupted and jumps to memory location 0023H to execute the ISR
- In that ISR we must examine the TI and RI flags to see which one caused the interrupt and respond accordingly
- The serial interrupt is used mainly for receiving data and is never used for sending data serially

To use the serial interrupt the ES bit along with the EA bit is set. Whenever one byte of data is sent or received, the serial interrupt is generated and the TI or RI flag goes high. Here, the TI or RI flag needs to be cleared explicitly in the interrupt routine (written for the Serial Interrupt).

The programming of the Serial Interrupt involves the following steps:

1. Enable the Serial Interrupt (configure the IE register).
2. Configure SCON register.
3. Write routine or function for the Serial Interrupt. The interrupt number is 4.
4. Clear the RI or TI flag within the routine.

Example 4

Write a program in which the 8051 reads data from P1 and writes it to P2 continuously while giving a copy of it to the serial COM port to be transferred serially. Assume that XTAL=11.0592MHz. Set the baud rate at 9600.

Solution:

```
ORG 0000H
LJMP MAIN

ORG 23H
LJMP SERIAL      ;jump to serial int ISR

ORG 30H
MAIN: MOV P1,#0FFH          ;make P1 an input port
MOV TMOD,#20H               ;timer 1, auto reload
MOV TH1,#0FDH               ;9600 baud rate
MOV SCON,#50H               ;8-bit,1 stop, ren enabled
MOV IE,10010000B            ;enable serial int.
SETB TR1                    ;start timer 1
BACK: MOV A,P1              ;read data from port 1
MOV SBUF,A                  ;give a copy to SBUF
MOV P2,A                    ;send it to P2
SJMP BACK                   ;stay in loop indefinitely
; SERIAL PORT ISR
```

```
ORG 100H
SERIAL: JB TI, TRANS        ;jump if TI is high
MOV A,SBUF                  ;otherwise due to receive
CLR RI                      ;clear RI
RETI                        ;return from ISR
TRANS: CLR TI               ;clear TI
RETI                        ;return from ISR
END
```

Example 5

Write a program in which the 8051 gets data from P1 and sends it to P2 continuously while incoming data from the serial port is sent to P0. Assume that XTAL=11.0592. Set the baud rata at 9600.

Solution:

```
ORG  0000H
LJMP MAIN

ORG  23H
LJMP SERIAL                 ;jump to serial int ISR

ORG  30H
MAIN: MOV  P1,#0FFH         ;make P1 an input port
MOV  TMOD,#20H   ;timer 1, auto reload
MOV  TH1,#0FDH              ;9600 baud rate
MOV SCON,#50H               ;8-bit,1 stop, ren enabled
MOV  IE,10010000B ;enable serial int.
SETB TR1                    ;start timer 1
BACK: MOV  A,P1             ;read data from port 1
MOV  P2,A                   ;send it to P2
SJMP BACK                   ;stay in loop indefinitely

;-----------------SERIAL PORT ISR
ORG 100H
SERIAL: JB  TI,TRANS        ;jump if TI is high
MOV A,SBUF                  ;otherwise due to receive
MOV P0,A                    ;send incoming data to P0
CLR RI                      ;clear RI since CPU doesn't
RETI                        ;return from ISR
TRANS: CLR TI               ;clear TI since CPU doesn't
RETI                        ;return from ISR
END
```

4.6 Interrupt Priority

When the 8051 is powered up, the priorities are assigned according to the following

Interrupt Priority Upon Reset

Highest To Lowest Priority	
External Interrupt 0	(INT0)
Timer Interrupt 0	(TF0)
External Interrupt 1	(INT1)
Timer Interrupt 1	(TF1)
Serial Communication	(RI + TI)

Example 5

Discuss what happens if interrupts INT0, TF0, and INT1 are activated at the same time. Assume priority levels were set by the power-up reset and the external hardware interrupts are edge triggered.

Solution:

When the above three interrupts are activated, IE0 (external interrupt 0) is serviced first, then timer 0 (TF0), and finally IE1 (external interrupt 1).

We can alter the sequence of interrupt priority by assigning a higher priority to any one of the interrupts by programming a register called IP (interrupt priority)

- To give a higher priority to any of the interrupts, we make the corresponding bit in the IP register high
- When two or more interrupt bits in the IP register are set to high, while these interrupts have a higher priority than others, they are serviced according to the sequence given in the above Table

Interrupt Priority Register (Bit-addressable)

D7							D0
--	--	PT2	PS	PT1	PX1	PT0	PX0

--	IP.7	Reserved
--	IP.6	Reserved
PT2	IP.5	Timer 2 interrupt priority bit (8052 only)
PS	IP.4	Serial port interrupt priority bit
PT1	IP.3	Timer 1 interrupt priority bit
PX1	IP.2	External interrupt 1 priority bit
PT0	IP.1	Timer 0 interrupt priority bit
PX0	IP.0	External interrupt 0 priority bit

Example 6

(a) Program the IP register to assign the highest priority to INT1(external interrupt 1), then
(b) discuss what happens if INT0, INT1, and TF0 are activated at the same time. Assume the interrupts are both edge-triggered.

Solution:

(a) MOV IP,#00000100B ;IP.2=1 assign INT1 higher priority.

The instruction SETB IP.2 also will do the same thing as the above line since IP is bit-addressable.

(b) The instruction in Step (a) assigned a higher priority to INT1 than the others; therefore, when INT0, INT1, and TF0 interrupts are activated at the same time, the 8051 services INT1 first, then it services INT0, then TF0. This is due to the fact that INT1 has a higher priority than the other two because of the instruction in Step (a). The instruction in Step (a) makes both the INT0 and TF0 bits in the IP register 0. As a result, the sequence in Table is followed which gives a higher priority to INT0 over TF0

Example 7

Assume that after reset, the interrupt priority is set the instruction MOV IP,#00001100B. Discuss the sequence in which the interrupts are serviced.

Solution:

The instruction "MOV IP #00001100B" (B is for binary) and timer 1 (TF1) to a higher priority level compared with the reset of the interrupts. However, since they are polled according to Table, they will have the following priority.

Highest Priority External Interrupt 1 (INT1)
Timer Interrupt 1 (TF1)
External Interrupt 0 (INT0)
Timer Interrupt 0 (TF0)
Lowest Priority Serial Communication (RI+TI)

4.7 Question Bank

1. Explain the structure of Interrupt priority and interrupt enable register.
2. Analyze Interrupt control used in 8051.
3. Explain how multiple interrupts are handled in 8051 microcontroller.
4. Define interrupt. List the steps involved in executing an interrupt.
5. Explain interrupt vector table of 8051 microcontroller.
6. Explain the steps involved in Programming Serial Communication Interrupts
7. Explain how interrupt priority can be changed using IP register. Also explain the default priorities assigned to interrupts in 8051 microcontroller.
8. Explain programming of timer interrupts.
9. Explain the following: i) Interrupt ii) Interrupt Service Routine iii)Interrupt Vector Table
10. Explain different types of interrupts in 8051 microcontroller.
11. Write a program in which the 8051 reads data from P1 and writes it to P2 continuously while giving a copy of it to the serial comport to be transferred serially. Assume that XTL = 11.0592 MHz Set the baud rate at 9600.
12. Write the instructions to:
    i) Enable the serial interrupt, Timer 0 interrupt and external hardware interrupt 1
    ii) Disable the Timer 0 interrupt
    iii) Disable all interrupts with a single instruction
13. Assume XTAL = 11.0592 MHz Use timer 0 to create the square wave. Write a program that continuously gets a 8-bit of data from P0 and sends it to P1, while simultaneously creating square wave of 200μs period on P2.5
14. Write a program to read the data from port P1 and send it to P2 continuously. While incoming data from the serial port is sent to P0. Assume XTAL = 11.0592 MHZ set the baud rate at 2400.
15. What is the use of IP register in 8051 microcontroller? If interrupts serial communication, Timer 1 and timer 2, are activated at the same time and if IP contains 10H then how the service will be provided to the interrupts.